

Model test 1:

LLM Front-End Framework Evaluation Report: Grado

Using step: WIN + R , type cmd

1.git clone <https://github.com/Gradio-App/gradio>

2.cd gradio

3.python demo/chatbot_multimodal/run.py

4.open web access [<http://127.0.0.1:7860>]

5.gradio/demo/chatbot_streaming/run.py

1. Overview

Framework Name: Grado

Positioning: A rapid AI/ML web interface building tool for Python developers.

Main Uses: For quickly building LLM dialogue consoles, hyperparameter adjustment panels, and multimodal prototype demos.

2. Core Advantages

* Extremely Low Coding Barrier (Python Native):

No need to write HTML/CSS/JavaScript; the entire UI layout and logic can be quickly built using pure Python code.

* Native Support for Large Model Dialogues and Streaming Output:

Built-in `gr.ChatInterface` and `gr.Chatbot` modules, providing out-of-the-box support for typewriter (streaming) effects, Markdown formatting, code highlighting, and one-click text copying.

Flexible Widget Rearrangement:

It provides the gr.Blocks layout system (Row / Column / Sidebar), allowing you to freely rearrange controls such as System Prompt, Temperature sliders, and Max Tokens input boxes in the sidebar or top.

Good API and SDK Compatibility:

It seamlessly integrates with the OpenAI Python SDK (such as the local LM Studio Server interface), making it very convenient to bind front-end interactive parameters (such as Temp=0.7) to back-end requests.

3. Disadvantages and Limitations: Lower Customization Limits Compared to a Full-fledged Product:

While styles support theme customization, the freedom to create highly customized CSS/front-end animations like those in the commercial versions of ChatGPT/Claude is less than with native React/Vue.

Weak in multi-user and state management:

More suitable for prototyping and demo demonstrations by a single person or internal team. Building a commercial web app with complex permission management, account login, and multiple historical session storage requires additional database expansion.

In summary, Gradio is Design Champion's best tool for rapidly delivering prototypes and validating interaction designs. For this project, it is recommended to use Gradio as the core front-end in the first phase (MVP phase) to showcase hardware limit test results, debug model hyperparameters, and demonstrate the complete API call chain to the team.

Siqi

03/08/2026

Here are a few evaluation reports for other popular LLM front-end frameworks, structured exactly like your reference format:

Model test 2:

LLM Front-End Framework Evaluation Report: Streamlit

Using step: WIN + R , type cmd

1. `pip install streamlit`
2. type `echo import streamlit as st > app.py` (or create `app.py` manually)
3. `python -m streamlit run app.py`
4. open web access [<http://localhost:8501>]

1. Overview

Framework Name: Streamlit

Positioning: A widely used open-source Python framework for turning data scripts into shareable web apps in minutes.

Main Uses: Building interactive data science dashboards, LLM dialogue interfaces, and internal data analysis tools.

2. Core Advantages

- **Data-Centric and Python Native:**

No front-end experience is needed. It integrates natively with Python's data ecosystem (Pandas, Numpy, Matplotlib), making it incredibly easy to display data tables and charts alongside LLM chats.

- **Built-in Chat Elements:**

Provides `st.chat_message` and `st.chat_input` widgets to easily construct ChatGPT-like UIs. Streaming is natively supported via the `st.write_stream` function.

- **Rich Component Ecosystem:**

A massive community provides hundreds of third-party components (Streamlit Components), allowing you to easily add custom features like audio recorders or specialized graphs.

- **Rapid Deployment:**

Offers Streamlit Community Cloud for free, one-click deployment directly from a GitHub repository.

3. Disadvantages and Limitations:

- **Execution Model Bottlenecks:**

Streamlit's architecture relies on running the *entire* Python script from top to bottom every time a user interacts with a widget. This can lead to performance issues in heavy applications and requires complex management of `st.session_state` to prevent data resets.

- **Rigid UI Customization:**

While it looks clean out of the box, altering the core layout or designing a highly bespoke UI/UX is notoriously difficult and usually requires injecting hacky custom CSS/HTML, which breaks the Python-only philosophy.

In summary, Streamlit is the best choice if your LLM application needs to heavily interact with databases, visualize data, or process files. For this project, if the next phase requires the AI to generate charts or analyze tabular data alongside the chat, Streamlit would be a strong upgrade over Gradio.

Siqi

06/08/2026

Model test 3:

LLM Front-End Framework Evaluation Report: Chainlit

Using step: WIN + R , type cmd

1. `pip install chainlit`
2. `type chainlit create-secret` (to generate auth secret if needed)
3. `type echo import chainlit as cl > app.py` (or create app.py manually)
4. `chainlit run app.py -w`

5. open web access [<http://localhost:8000>]

1. Overview

Framework Name: Chainlit

Positioning: An open-source Python package specifically designed to build production-ready Conversational AI applications.

Main Uses: Creating ChatGPT-like UIs, multi-agent conversation hubs, and visually debugging complex LangChain/LlamaIndex execution workflows.

2. Core Advantages

- **Purpose-Built for LLMs:**

Unlike Gradio or Streamlit which are general-purpose tools, Chainlit is built from the ground up specifically for chat interfaces.

- **"Chain of Thought" Visualization:**

Native integration with LangChain, LlamaIndex, and AutoGen. It allows developers and users to expand chat bubbles to see intermediate steps, prompts, and tool uses (like web searches or code execution) directly in the UI.

- **Enterprise Features Out-of-the-Box:**

Provides built-in support for multi-user authentication, data persistence (saving chat histories to databases), and user feedback mechanisms (thumbs up/down on AI messages).

- **Rich Media and Multi-Modal:**

Excellent built-in support for file uploads, audio, video, and image generation within the chat stream.

3. Disadvantages and Limitations:

- **Steeper Learning Curve:**

Because it relies heavily on Python decorators (e.g., `@cl.on_message`) and asynchronous programming (`async/await`), it requires a slightly deeper understanding of Python than Gradio's simple scripting approach.

- **Narrower Focus:**

It is highly specialized for chat. If the project requires non-conversational AI dashboards or standard data visualization interfaces, Chainlit is not the right tool.

In summary, Chainlit is the optimal choice for applications utilizing complex agent frameworks like LangChain. If the project evolves to require multi-agent workflows, tool-calling visualization, or robust user session management, Chainlit should replace Gradio in the production phase.

Siqi

06/08/2026